# Sudokus mit dem Paket `sudoku` setzen

Fortbildungsmaterial für MINT-Lehrkräfte

11. November 2026

Sudokus sind beliebt für Vertretungsstunden, als Knobelaufgabe zum Stundenende oder als Einstieg in das Backtracking im Informatikunterricht. Mit dem Paket `sudoku` setzt LaTeX die Gitter sauber und einheitlich. Schwierig ist etwas anderes: Ein selbst ausgedachtes Sudoku hat fast nie genau eine Lösung. Diese Handreichung zeigt deshalb zuerst, wie man Gitter setzt und gestaltet, und dann, wie ein Python-Skript neue Rätsel samt Lösung erzeugt und vorhandene Rätsel prüft.

## 1 Grundlagen und benötigte Pakete

In der universalen `header.tex` ist das Paket bereits geladen. In einem eigenständigen Dokument genügt:

```
\usepackage{sudoku}
```

Das Paket `sudoku` darf nicht zusammen mit `printsudoku` oder `sudokubundle` geladen werden. Beide definieren ebenfalls einen Befehl `\sudoku`, und die Fehlermeldung ist dann kaum zu verstehen.

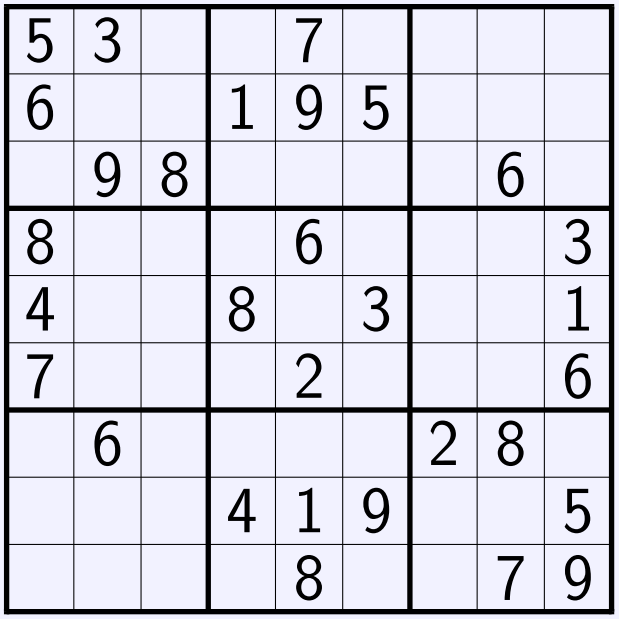
### 1.1 Ein erstes Gitter

Ein Sudoku steht in der Umgebung `sudoku`. Jede der neun Zeilen beginnt mit einem senkrechten Strich, die Zellen werden durch `|` getrennt, und am Ende jeder Zeile steht ein Punkt. Eine leere Zelle ist ein Leerzeichen.

```
\begin{sudoku}
|5|3| | |7| | | | |.
|6| | |1|9|5| | | |.
| |9|8| | | | |6| |.
|8| | | |6| | | |3|.
|4| | |8| |3| | |1|.
```

```
|7| | | |2| | | |6|.
| |6| | | | |2|8| |.
| | | |4|1|9| | |5|.
| | | | |8| | |7|9|.
\end{sudoku}
```

**Ausgabe**



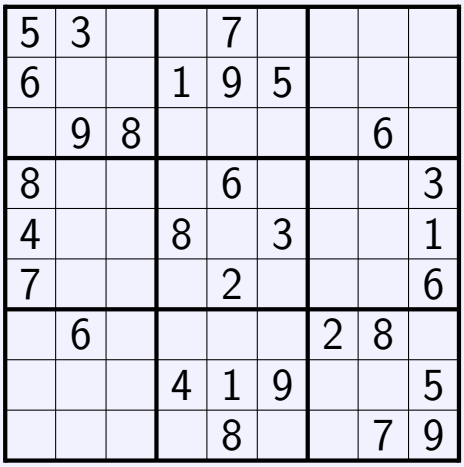

Ohne weitere Angabe ist das Gitter 10 cm breit und die Ziffern stehen in der Schriftgröße `\Huge`; in der Ausgabe oben ist das Gitter auf 8 cm verkleinert (siehe nächstes Kapitel). Für kleinere Gitter sollte man auch die Schrift verkleinern. Fehlt am Zeilenende der Punkt, verrutscht das ganze Gitter. Deshalb darf auch in einer Zelle kein Punkt stehen.

## 2 Größe, Linien und Schrift

Das Aussehen wird über drei Längen und einen Befehl gesteuert: `\sudokusize` ist die Kantenlänge, `\sudokuthinline` und `\sudokuthickline` sind die Strichstärken innerhalb und zwischen den 3×3-Blöcken, `\sudokuformat` legt die Schrift jeder Zelle fest.

```
\setlength\sudokusize{6cm}
\setlength\sudokuthinline{0.4pt}
\setlength\sudokuthickline{1.6pt}
\renewcommand*\sudokuformat[1]{\Large\sffamily#1}
\begin{sudoku}
|5|3| | |7| | | | |.
|6| | |1|9|5| | | |.
| |9|8| | | | |6| |.
|8| | | |6| | | |3|.
|4| | |8| |3| | |1|.
|7| | | |2| | | |6|.
| |6| | | | |2|8| |.
| | | |4|1|9| | |5|.
| | | | |8| | |7|9|.
\end{sudoku}
```

**Ausgabe**




Die Einstellungen gelten bis zum Ende der aktuellen Gruppe. Stehen sie in der Präambel, gelten sie für alle Sudokus des Dokuments, so sehen alle Rätsel eines Arbeitsblatts gleich aus.

## 3 Zellen hervorheben

In eine Zelle darf auch ein Befehl geschrieben werden, etwa für Fettdruck oder Farbe. Dann muss der Inhalt der Zelle in geschweiften Klammern stehen: `|{\textbf{5}}|`. Ohne Klammern liest das Trennzeichen `|` nur das erste Zeichen, und LaTeX bricht mit „Argument of \textcolor has an extra }“ ab.

Das Beispiel zeigt eine Lösung, in der die Vorgaben fett und die ergänzten Ziffern grau gesetzt sind. Die mittlere Zeile ist zusätzlich rot markiert:

```
\setlength\sudokusize{6cm}
\renewcommand*\sudokuformat[1]{\Large\sffamily#1}
\newcommand\vor[1]{{\textbf{#1}}}
\newcommand\erg[1]{{\textcolor{black!45}{#1}}}
\newcommand\rot[1]{{\textcolor{red}{\textbf{#1}}}}
\begin{sudoku}
|{\vor5}|{\vor3}|{\erg4}|{\erg6}|{\vor7}|{\erg8}|{\erg9}|{\erg1}|{\erg2}|.
|{\vor6}|{\erg7}|{\erg2}|{\vor1}|{\vor9}|{\vor5}|{\erg3}|{\erg4}|{\erg8}|.
|{\erg1}|{\vor9}|{\vor8}|{\erg3}|{\erg4}|{\erg2}|{\erg5}|{\vor6}|{\erg7}|.
|{\vor8}|{\erg5}|{\erg9}|{\erg7}|{\vor6}|{\erg1}|{\erg4}|{\erg2}|{\vor3}|.
|{\rot4}|{\rot2}|{\rot6}|{\rot8}|{\rot5}|{\rot3}|{\rot7}|{\rot9}|{\rot1}|.
|{\vor7}|{\erg1}|{\erg3}|{\erg9}|{\vor2}|{\erg4}|{\erg8}|{\erg5}|{\vor6}|.
|{\erg9}|{\vor6}|{\erg1}|{\erg5}|{\erg3}|{\erg7}|{\vor2}|{\vor8}|{\erg4}|.
|{\erg2}|{\erg8}|{\erg7}|{\vor4}|{\vor1}|{\vor9}|{\erg6}|{\erg3}|{\vor5}|.
|{\erg3}|{\erg4}|{\erg5}|{\erg2}|{\vor8}|{\erg6}|{\erg1}|{\vor7}|{\vor9}|.
\end{sudoku}
```

**Ausgabe**

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 5 | 3 | 4 | 6 | 7 | 8 | 9 | 1 | 2 |
| 6 | 7 | 2 | 1 | 9 | 5 | 3 | 4 | 8 |
| 1 | 9 | 8 | 3 | 4 | 2 | 5 | 6 | 7 |
| 8 | 5 | 9 | 7 | 6 | 1 | 4 | 2 | 3 |
| 4 | 2 | 6 | 8 | 5 | 3 | 7 | 9 | 1 |
| 7 | 1 | 3 | 9 | 2 | 4 | 8 | 5 | 6 |
| 9 | 6 | 1 | 5 | 3 | 7 | 2 | 8 | 4 |
| 2 | 8 | 7 | 4 | 1 | 9 | 6 | 3 | 5 |
| 3 | 4 | 5 | 2 | 8 | 6 | 1 | 7 | 9 |

Die Hilfsbefehle `\vor`, `\erg` und `\rot` halten das Gitter kurz. Auch sie müssen in der Zelle in Klammern stehen: `|{\vor5}|`.

# 4 Neue Rätsel erzeugen

## 4.1 Der Generator

Neue Sudokus erzeugt das Python-Skript `sudoku_generator.py` aus dem Ordner `scripts` des Skills. Es füllt zuerst ein vollständiges Gitter und streicht dann so lange Ziffern, bis die gewünschte Schwierigkeit erreicht ist. Nach jedem Streichen prüft es per Backtracking, ob es noch genau eine Lösung gibt. Der Aufruf im Terminal:

```
python3 sudoku_generator.py -n 2 -s mittel --seed 7 -o beispiel_sudokus.tex
```

- `-n 2` erzeugt zwei Rätsel.
- `-s leicht`, `mittel` oder `schwer` legt die Zahl der Vorgaben fest (etwa 40, 32 oder 26).
- `--seed 7` macht das Ergebnis wiederholbar: Mit derselben Zahl entstehen dieselben Rätsel. Für Parallelklassen nimmt man verschiedene Zahlen.
- `-o` gibt die Zieldatei an.

## 4.2 Die erzeugten Rätsel einbinden

Die erzeugte Datei enthält für jedes Rätsel zwei Befehle: `\SudokuAufgabeA` und `\SudokuLoesungA`, dann `\SudokuAufgabeB` und so weiter. Man lädt sie in der Präambel:

```
\input{beispiel_sudokus.tex}
```

Danach lassen sich die Rätsel überall aufrufen, zum Beispiel nebeneinander in zwei `minipage`-Umgebungen:

```
\setlength\sudokusize{6.5cm}
\renewcommand*\sudokuformat[1]{\Large\sffamily#1}
\begin{minipage}[t]{0.48\linewidth}\centering
  \textbf{Aufgabe}
  \SudokuAufgabeA
\end{minipage}\hfill
\begin{minipage}[t]{0.48\linewidth}\centering
  \textbf{Lösung}
  \SudokuLoesungA
\end{minipage}
```

**Ausgabe**

**Aufgabe**

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 2 | | | 3 | 6 | | 1 | 9 | |
| 3 | | | | 9 | | | | |
| | 8 | 1 | | | | 3 | | 7 |
| | | 2 | 8 | | | | | |
| 7 | | | 9 | 3 | 6 | | | 2 |
| | | | | | 1 | 8 | | |
| 4 | | 5 | | | | 2 | 8 | |
| | | | | 5 | | | | 6 |
| | 3 | 9 | | 8 | 2 | | | 5 |

**Lösung**

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| **2** | 5 | 4 | **3** | **6** | 7 | **1** | **9** | 8 |
| **3** | 7 | 6 | 1 | **9** | 8 | 5 | 2 | 4 |
| 9 | **8** | **1** | 2 | 4 | 5 | **3** | 6 | **7** |
| 5 | 9 | **2** | **8** | 7 | 4 | 6 | 3 | 1 |
| **7** | 1 | 8 | **9** | **3** | **6** | 4 | 5 | **2** |
| 6 | 4 | 3 | 5 | 2 | **1** | **8** | 7 | 9 |
| **4** | 6 | **5** | 7 | 1 | 9 | **2** | **8** | 3 |
| 8 | 2 | 7 | 4 | **5** | 3 | 9 | 1 | **6** |
| 1 | **3** | **9** | 6 | **8** | **2** | 7 | 4 | **5** |

In der Lösung stehen die Vorgaben fett und die ergänzten Ziffern grau, so findet man beim Kontrollieren schnell die Stelle, an der ein Fehler liegt. Im Ordner `assets` des Skills liegt ein fertiges Arbeitsblatt mit vier Rätseln auf der Vorderseite und den Lösungen auf der Rückseite. Die Datei heißt `vorlage_sudoku_arbeitsblatt.tex`.

## 4.3 Ein vorhandenes Sudoku prüfen

Sudokus aus Zeitungen oder Rätselheften lassen sich mit demselben Skript lösen und prüfen. Dazu schreibt man das Gitter zeilenweise als 81 Zeichen auf, eine leere Zelle ist ein Punkt:

```
python3 sudoku_generator.py --loesen
    "53..7....6..195....98....6.8...6...34..8.3..17...2...6.6....28....419..5....8..79"
```

Das Skript gibt den LaTeX-Code für Aufgabe und Lösung aus und warnt, wenn das Rätsel mehr als eine Lösung hat. Das ist bei selbst ausgedachten Sudokus fast immer der Fall.

# 5 Mini-Sudokus mit Ti*k*Z

Das Paket kennt nur das $9 \times 9$-Gitter. Für die Klassen 5 und 6 eignen sich kleinere Sudokus mit $4 \times 4$ Feldern und den Ziffern 1 bis 4. Sie lassen sich mit Ti*k*Z zeichnen: ein dünnes Gitter, darüber dicke Linien für die $2 \times 2$-Blöcke, dann die Vorgaben.

```
\begin{tikzpicture}[scale=0.9]
  \draw[step=1, thin] (0,0) grid (4,4);
  \draw[very thick] (0,0) rectangle (4,4) (2,0) -- (2,4) (0,2) -- (4,2);
  \foreach \x/\y/\z in {0/3/1, 1/3/2, 0/2/3, 3/2/2, 0/1/2, 3/1/3, 2/0/2}
    \node[font=\Large\sffamily] at (\x+0.5,\y+0.5) {\z};
\end{tikzpicture}
```

**Ausgabe**

| | | | |
|---|---|---|---|
| 1 | 2 | | |
| 3 | | | 2 |
| 2 | | | 3 |
| | | 2 | |

In der Liste stehen Spalte, Zeile (von unten gezählt) und Ziffer. Dieses Rätsel hat genau eine Lösung, das wurde durch Ausprobieren aller Möglichkeiten geprüft. Bei eigenen Mini-Sudokus muss man die Eindeutigkeit selbst prüfen, denn der Generator arbeitet nur mit $9 \times 9$-Gittern.

# 6 Zusammenfassung

Die wichtigsten Techniken im Überblick:

- Ein Gitter steht in `\begin{sudoku}` … `\end{sudoku}`: neun Zeilen, Zellen durch `|` getrennt, Punkt am Zeilenende.
- Größe, Linien und Schrift mit `\sudokusize`, `\sudokuthinline`, `\sudokuthickline` und `\sudokuformat`.
- Befehle in einer Zelle immer in geschweifte Klammern setzen.
- Rätsel nicht selbst ausdenken, sondern mit `sudoku_generator.py` erzeugen; vorhandene Rätsel mit `--loesen` prüfen.
- Für $4 \times 4$-Sudokus ein Gitter mit Ti*k*Z zeichnen und die Eindeutigkeit selbst prüfen.

Im Informatikunterricht ist das Sudoku ein anschauliches Beispiel für Backtracking: Die Funktion `loese()` im Generator ist kurz genug, um sie gemeinsam zu lesen. Eine gute Frage dazu ist, warum es so viel schneller geht, wenn man immer zuerst das Feld mit den wenigsten möglichen Ziffern ausfüllt. In der Mathematik passt die Frage, wie viele Vorgaben ein eindeutiges Sudoku mindestens braucht. Die Antwort 17 wurde erst 2012 mit Computerhilfe bewiesen.